

Unit-3

Creating Types in C#



Class task

- Create a class StudentModel under a nested namespace Student->Model
- Member Fields:
 - Id[int,readonly] ;
 - name[string] ;
 - marks[int]
- Add a constructor that takes: id, name, and marks as input Sets them to the fields.
- Add these methods
 - DisplayInfo() → print the student details
 - UpdateMarks(ref int newMarks) → uses ref to update marks
 - IsPassed() → return true if marks are 40 or more

- Classes & Objects
 - Constructor
 - Deconstructor
 - this reference
- Properties
- Indexers
- Modifiers
 - Access Modifier
 - Static/keyword Modifier
- Inheritance
- Overloading
- Dynamic Binding/Overriding
- base keywords
- Abstract class & Methods
- Interface
- Generics

Classes & Object:

A class is a blueprint or template that defines the structure and behaviour (data and actions) of objects.

An object is a concrete instance of a class or real world representation of classes.

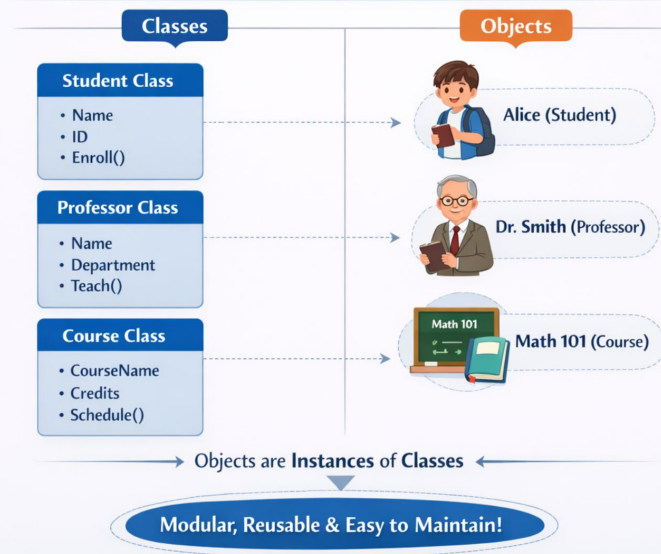
While a class defines the structure and behaviour (the blueprint), an object is an actual entity that exists in memory and can be used in the program.

Class Members:

- **Member Fields**
- **Methods**
- **Constructor**
- **Destructor**
- **Properties**
- **Indexers**
- **Events**
- **Operators**

Object-Oriented Programming (OOP) in C#

OOP is a programming paradigm that organizes code using objects and classes. It helps make programs **modular**, **reusable**, and **easier to maintain**.



Member Fields

These are variables declared inside a class, used to store data or state.

static Modifier [static]

Access Modifier [public, private, protected, internal]

Inheritance Modifier [new]

readonly Modifier [readonly]

```
class Car
```

```
{  
    public static string brand;  
    private int speed;  
}
```

Readonly Modifier

Prevents field from being modified

Only be assigned:

- at the time of declaration, or
- inside a constructor

Member Methods

Functions inside a class — define behavior.

```
[...modifier] [returnType] [methodName] ([...parameters])  
{  
    // method body (statements to execute)  
}
```

Types of Modifier Supported

- Static Modifier [static]
- Access Modifier [public,private,protected,internal]
- Inheritance Modifier [new,virtual,abstract,override,sealed]
- Partial method modifier [partial]
- Async Code Modifier [async, async*]

```
class Car
{
    public static void Drive()
    {
        Console.WriteLine("Driving...");
    }

    public partial void Drive1()
    {
        Console.WriteLine("Driving...");
    }

    private int CalculateSpeed()
    {
        return 100;
    }
}
```


Constructor

A constructor is a method in class

- the same name as the class and
- does not have a return type.
- Limited Modifier.
- Invoked automatically when an object is created with new

A single class can have multiple constructor

Use: is a special method used to initialize an object.

Modifier: static Modifier, Access Modifier

Types of Constructor

- Default Constructor
- Parameterized Constructor
- Copy Constructor
- Static Constructor
- Private Constructor
- Constructor Chaining

1. Default Constructor

A Constructor without any parameter.

Can be built-in/user defined.

Use:

- Allows object creation without passing arguments
- Sets Member fields with default value.

2. Parameterized Constructor

A Constructor with parameters.

Must be user defined.

Use:

- Allows you to set values at the time of object creation.

```
class Car
{
    public static string brand;
    private int speed;
    // Parameterized Constructor
    public Car(string carBrand, int initialSpeed)
    {
        brand = carBrand;
        speed = initialSpeed;
    }
    public void ShowInfo()
    {
        Console.WriteLine($"Brand: {brand}, Speed: {speed}");
    }
}
```

3. Copy Constructor

A Constructor with object as an parameter.

Not built in: must be explicitly defined.

Use:

- Used to create a new object by copying values from an existing object.

```
class Car
{
    public string Brand;
    public int Speed;

    // Copy constructor
    public Car(Car existingCar)
    {
        Brand = existingCar.Brand;
        Speed = existingCar.Speed;
    }

    public void ShowInfo()
    {
        Console.WriteLine($"Brand: {Brand}, Speed: {Speed}");
    }
}
```

4. Static Constructor

- Called only once before any **object is created** or **any static member** is accessed.
- Doesn't matter how many time the instance is created.
- It will be invoked first by CLR itself
- Won't accept any parameter and access modifier

Use:

- For initializing static member fields.

```
class DatabaseConfig
{
    public static string ConnectionString;

    // Static constructor
    static DatabaseConfig()
    {
        Console.WriteLine("Static constructor called");
        ConnectionString = "Server=localhost;Database=MyApp;";
    }
}
```

```
class Car
{
    public static string brand;
    private int speed;
    // Static Constructor
    static Car()
    {
        brand = "Default Brand";
    }

    public Car(int initialSpeed)
    {
        speed = initialSpeed;
    }

    public void ShowInfo()
    {
        Console.WriteLine($"Brand: {brand}, Speed: {speed}");
    }
}
```

Class Task

Create a class Laptop that includes:

- Fields: manufacturer (static), ramSize, modelName, serialNumber (readonly)
- Constructor
 - Static constructor (sets manufacturer)
 - Default Constructor
 - Parameterized Constructor
 - Copy Constructor
- Methods:
 - ShowSpecs() – displays RAM and model
 - DisplayDetails() – displays all info
 - ShowManufacturer() – static method

Private Constructor

is a constructor that can only be called within the same class.

```
class Car
{
    private Car()
    {
        // Constructor logic here
    }
}
```

Commonly used in Singleton Pattern

Ensures only one instance of a class exists.

```
public class Singleton {  
    private static Singleton instance;  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null)  
            instance = new Singleton();  
        return instance;  
    }  
}
```

Destructor

A destructor is a special method in C# that's used to clean up when an object is being destroyed and garbage-collected.

No parameters, no return type, no access modifier.

Syntax:

```
~ClassName ()  
{  
    // Cleanup code here  
}
```

Use Case:

Commonly used in cleaning Up Unmanaged Resources (as a fallback)

```
class FileHandler
{
    private FileStream file;

    public FileHandler(string path)
    {
        file = new FileStream(path, FileMode.OpenOrCreate);
        Console.WriteLine("File opened.");
    }

    ~FileHandler()
    {
        file?.Close();
        Console.WriteLine("File closed in destructor.");
    }
}
```

When is Destructor Called?

Called after an **object becomes unreachable**.

The **GC decides** when to call the destructor — **not the programmer**.

Timing is **non-deterministic**.

```
GC.Collect();
```

```
GC.WaitForPendingFinalizers();
```

Class Task

1. Create a Car class with a private constructor.
2. Implement a static method CreateCar() to create and return a new Car object.
3. Add a destructor to simulate cleaning up resources (e.g., closing a file or database connection).
4. In Main(), create two Car objects using CreateCar(), confirm they are the same instance, set them to null, and trigger garbage collection to call the destructor.

Properties

are special members of a class that provide a flexible mechanism to read, write, or compute the values of private fields.

Syntax:

```
public class Person
{
    private string name;    // Private field

    public string Name      // Public property
    {
        get { return name; }
        set { name = value; }
    }
}
```

Auto Implemented Properties

```
public class Person
{
    public string Name { get; set; } // Auto-implemented property
}
```

Read-only Properties

```
public string Name { get; private set; } // Read-only from outside
```


Class Task

Class Name: BankAccount

Private fields:

- accountNumber (read/write)
- accountHolder (read/write)
- balance (read)

Public methods:

- Deposit(double amount)
- Withdraw(double amount)

this reference

- **Resolving Name Conflicts**

If a method parameter has the same name as a class field, `this` helps differentiate them.

class Person

```
{  
    private string name;  
  
    public void SetName(string name)  
    {  
        this.name = name; // 'this.name' refers to the field, 'name' is the parameter  
    }  
}
```

- **Passing current object as parameter**
- **Constructor Chaining[this]**
- **Indexer**

Constructor Chaining

Constructor Chaining is when one constructor calls another in the same class using the `this` keyword.

```
class MyClass
{
    public MyClass() : this("roshni")
    {
        // optional code
    }

    public MyClass(string value)
    {
        // constructor logic
    }
}
```

Class Task

Create a Student class with:

Name and Grade properties.

- Constructor1= that takes name and grade.
- Constructor2 that takes only name[sets grade to "Not Assigned" using constructor chaining.]
- A method to display student info.

Indexer

An indexer allows an object to be indexed like an array.

A class can have more than one indexer but of differ in numbers or type of parameters.

```
public class ClassName
{
    private Type[] items = new Type[Size];

    public Type this[int index]
    {
        get { return items[index]; }
        set { items[index] = value; }
    }
}
```

Use Case:

- Access object like an array
- Hide internal data structure
- Can be use to create custom collection classes

Create a class named StudentList that stores names of 5 students in an array.
Use a private string array inside the class.

Create an indexer to:

- set student names
- get student names

Class Task

Create a Student class with:

Name and Grade properties.

- Constructor1= that takes name and grade.
- Constructor2 that takes only name[sets grade to "Not Assigned" using constructor chaining.]
- A method to display student info.

Modifer

Specials keywords added during declaration that modify/alter the behaviour of members in a class.

- **Static Modifier**
- **Access Modifier**
- **Class Modifier**
- **Inheritance Modifier**

Static Modifier/keyword

Use to defined member that belongs to type itself rather than a specific instance.

Usage

- Static class
- Static member field
- Static method
- Static Constructor
- Static properties

Static Class

- cannot be instantiated
- can only contain static members.
- Is sealed by default (cannot be inherited)

Static Member Field

- Accessed through class name, not instance
- Single copy shared by all instances

Static Method

- Accessed through class name, not instance
- Can only access static members
- Cannot use this reference

Static Constructor

- initializes static members of a class.
- Executes exactly once, [before any static members are accessed or instances are created]
- Cannot be called directly[on demand]
- No access modifiers or parameters

Class Task

Create a class Student:

Static Fields:

- SchoolName (string): The name of the school (shared by all students)
- TotalStudents (int): Tracks the total number of students enrolled

Static Constructor:

- Initialize the school name to "Nepathya College"
- Initialize the student counter to 0 Print a welcome message when the system initializes

Member Fields[Instance Field]:

- Name(String), studentId(int), GPA(double)

Member Method[Instance Method]

- DisplayInfo(): Shows student details

Parametrized Constructor[increase TotalStudents by one]

Static Method:-

- DisplaySchoolInfo(): Shows school name and total students

Access Modifier/keyword

Use to specify/control visibility/accessibility of a type(classes,struct,interface) and it's member

- **Public**
 - Accessible from anywhere (no restrictions).
- **Private**
 - Accessible only within the same class.
- **Protected**
 - Accessible within the same class and derived (child) classes.
- **internal**

By Default class members are **private**

By Default class, struct are **internal**

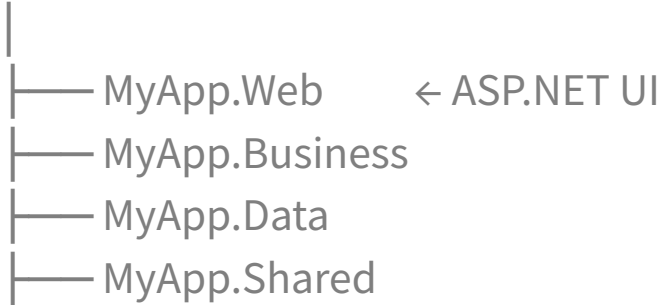
Internal

keyword restricts access to the current assembly (project).

It is commonly used for types or members that should be accessible only within the same project (DLL or EXE) but hidden from other projects that reference it.

Use of internal modifier

MySolution



Inheritance

A class acquires the properties and behaviors (methods) of another class.
Promotes code reusability and relationships between classes.

Syntax:

```
class Parent
{
    // parent members
}
```

```
class Child : Parent
{
    // child members
}
```

Types of Inheritance

Single Inheritance

One child class inherits from one parent class.

Syntax:

```
class Animal { }  
class Dog : Animal { }
```

Multilevel Inheritance

A class inherits from a child class (a chain).

Syntax:

```
class Animal { }           //Grand Parent  
class Dog : Animal { }     //Parent  
class Puppy : Dog { }      //Child
```


Hierarchical Inheritance

Multiple classes inherit from the same parent class.

Syntax:

```
class Animal { }  
class Dog : Animal { }  
class Cat : Animal { }
```

Multiple Inheritance (via Interfaces)

A class inherits from multiple interfaces, not multiple classes.

Syntax:

```
interface IAnimal { }  
interface IPet { }  
class Dog : IAnimal, IPet { }
```

Class Task

Create a Vehicle Management System.

- ElectricCar name, [Start, Charge, Drive]
- PetrolCar name, [Start, FillPetrol, Drive]
- Bike name, [Start, Ride]
- Truck name, [Start, LoadGoods,]

(Insurance, Maintenance) Common interface for All Class

Polymorphism

Same method name, but different behaviors based on the object calling it.

Types:

- Overloading/Compile Time Polymorphism/Static Polymorphism
- Overriding/ RunTime Polymorphism/ Dynamic Polymorphism

Method Overloading/Compile Time Polymorphism/Static Polymorphism

multiple methods with the same name but different signatures (different number, type, or order of parameters) within the same class.

Overloading can happen within the same class or in inheritance hierarchies.

Why called Compile Time Polymorphism?

The compiler decides which version of the method to call based on arguments.

Method Signature

uniquely identifies a method within a class with a combination of a name and the type, number, and order of its parameters.

- Name
- Number of parameter
- Parameter type
- Parameter Ordering

Write a C# program with a class **BillCalculator** that overloads the CalculateBill method in three ways:

- Calculate bill for one item (price).
- Calculate bill for multiple items (price, quantity).
- Calculate bill for multiple items with discount (price, quantity, discount).

Overriding/ RunTime Polymorphism/ Dynamic Polymorphism

a feature that allows a derived class to provide its own implementation of a method that is already defined in its base class.

The method in the base class must be marked as virtual or abstract
the method in the derived class must use the override keyword.

Inheritance modifier/Keywords

virtual

- ❖ used in the base class to declare a method that can be overridden by a derived class.
- ❖ base method will have a default behavior, but derived classes can change it by overriding it.

Override

- ❖ is used in a derived class to change the behavior of a virtual or abstract method defined in a base class.
- ❖ provides a new implementation for the method.

Class Task

Create a Vehicle Management System.

- ElectricCar [(modelName),Start, Charge,Drive,DisplayInfo]
- PetrolCar[(modelName),Start,FillPetrol,Drive,DisplayInfo]
- Bike [(modelName),Start,Ride,DisplayInfo]
- Truck [(modelName),Start, LoadGoods,,DisplayInfo]

(Insurance,Maintainance) Common interface for All Class

- Override the method Start in all derived Classes
- Hide Method DisplayInfo() [using new keyword]
- Use base constructor chaining to pass modelName from Derived class

Method Hiding

defines a method with the same name as a method in its base class, but does not override it.

Instead it hides it

introduces a **new** method that hides the base method.

The decision about which method to call is made based on the reference type at compile time, NOT the runtime object.

Base keyword

base keyword is used to access members (methods, properties, constructors) of a parent (base) class from within a derived (child) class.

- Calling a base class constructor
- Calling a base class method (overridden)
- Accessing a base class property

dynamic keyword

dynamic keyword is used to declare a variable whose type is resolved at runtime instead of compile-time.

```
dynamic something = 5;
```

```
Console.WriteLine(something); // Outputs 5
```

```
something = "Hello";
```

```
Console.WriteLine(something); // Outputs Hello
```

```
something = new List<int> { 1, 2, 3 };
```

```
Console.WriteLine(something.Count); // Works at runtime
```

Abstract class

A class that cannot be instantiated.

Declared using the abstract keyword.

Meant to be inherited by other classes.

Can contains both:

- Abstract members[Access modifier]
- Non- Abstract member

Method, properties, indexers but not member fields

```
public abstract class Animal
{
    public abstract void MakeSound();    // No implementation
    public void Eat() => Console.WriteLine("Eating");
}
```

Class Task

WAP to to create abstract StudentBase:-

- Properties(Name[string])
- Method(void DisplayInfo())
- Indexer for storing courses [string this[int index]] along with get/set.

Implement this Abstractin class Student

Interface

An interface defines a [set of methods, properties, indexer, events etc.] that implementing classes must fulfill.

Declared using the interface keyword.

Unlike Contains only method/property signatures [No Implementation].

No Access Modifier

Interface support Multiple Inheritance

```
public interface IAnimal
{
    void MakeSound();
}
```

Class Task

WAP to create an interface IAttendance:

Methods:

- void MarkAttendance()
- void ViewAttendance()

Implement this interface in:

- Lecturer
- Student

Operator keyword/ Operator Overloading

Operators are symbols like +, -, == used to perform operations.

In a class, you can overload operators to work with custom implementation.

```
public static ReturnType operator OperatorSymbol (Type operand1, Type operand2)
{
    // implementation
}
```

Only certain operators can be overloaded.

Must use static methods inside class/struct.

Class Task

WAP to create a class Money with Properties Amount[decimal] & currency

Create a Parametized Constructor to pass amount and currency

Perform Operator Overloading:

- + [Take two instance of Money & perform sum of Amount , return type Money]
- - [Take two instance of Money & perform subtraction of Amount, return type Money]

Class Task

WAP to create a class Student with Properties Marks[double] & Name

Create a Parametized Constructor to pass Marks and Name

Perform Operator Overloading:

- + [Take two instance of Student & perform sum of Marks , return type double
;sumOfMarks]
- ==Take two instance of Student & perform comparison of Marks, return type bool]

Generic

Allow definition of classes, methods, interfaces with a placeholder type without specifying a real data type.

Use `<T>(TypeOfValue)` as a placeholder.

Purpose:

This enables type safety without committing to a specific data type up front.

1. User Defined Generic
2. Predefined Generic Classes

User Defined Generic

1. Generic Class

```
public class ClassName<T>
{
    public T name;
}
```

2. Generic Method

```
public void MethodName<T>(T param)
{
    // ...
}
```

3. Generic Interface

```
public interface IInterfaceName<T>
{
    T Get(int id);
}
```

Pre-defined Generic Collection Class

1. List<T>
2. Dictionary<TKey, TValue>
3. Queue<T>
4. Stack<T>

Class Task

Create a class called `Box<T>` that can store or retrieve value

1. Member Field:
 - a. `_item[private, T]`
2. Member Method
 - a. `SetValue[SetValue(T item), return type : void ]`
 - b. `GetValue[T Get()], return type T]`

Class Task

Create a class called DataStore<T> with Crud Operation

1. Member Field:
 - a. `_items[private, List<T>]`
2. Member Method
 - a. `AddItem [Add(T item), return type : void ]`
 - b. `GetItem[T Get(int index), return type T]`
 - c. `GetAllItem(List<T> GetAll(),return type : List<T> )`

Define a class Product with properties Name(string), price(double)

Create 5 instance of Product & Use DataStore to store those Products